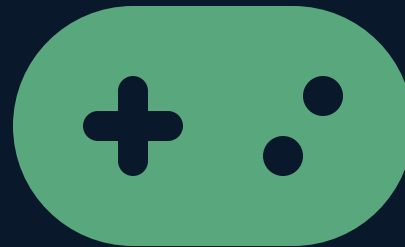


PORTADA

Análisis y Diseño de Sistemas II

Semana 10 — Patrones GRASP: Controller, Low Coupling y High Cohesion

Universidad de Antioquía · Ingeniería de Sistemas



APERTURA

**¿Quién debería recibir el clic —
el botón, o la clase que representa el caso de uso?**

TRANSICIÓN

Seguimos la serie de patrones GRASP

El martes vimos los dos primeros patrones de **GRASP** (*General Responsibility Assignment Software Patterns*, el catálogo de Craig Larman para decidir en qué clase recae cada responsabilidad): **Information Expert** (asignarle la responsabilidad a la clase que tiene la información necesaria para cumplirla) y **Creator** (quién debe instanciar un objeto nuevo).

Hoy vemos **tres patrones más**: **Controller** (quién recibe un evento de la interfaz), **Low Coupling** y **High Cohesion** — estos dos últimos ya los conocen como conceptos de la semana 7, hoy se formalizan como patrones GRASP con criterios concretos para evaluarlos en un diseño.

CONCEPTO

El patrón Controller

Un **evento de sistema** es una entrada externa que un actor genera sobre el sistema (un clic, un envío de formulario, una llamada a una API) — es el mismo concepto de los **Diagramas de Secuencia del Sistema** de la semana 4, donde el sistema se trataba como una caja negra que recibía estos eventos.

Controller (GRASP): asigna la responsabilidad de **manejar un evento de sistema** a una clase que representa una de estas tres cosas:

- El **sistema completo** o un subsistema.
- Un **caso de uso** específico (o un escenario de ese caso de uso).
- Una **sesión** de un actor con el sistema.

La regla clave: **nunca la interfaz de usuario misma** (el botón, la ventana, el endpoint HTTP). La UI recibe el evento y se lo delega al Controller; no decide qué hacer con él.

CONCEPTO

Dos variantes de Controller

Variante	Cuándo usarla	Ejemplo
Controller de caso de uso	Un controller por cada caso de uso (o familia muy cercana de casos de uso)	RegistrarPedidoController , CancelarPedidoController
Controller de fachada (sesión/subsistema)	Un controller que agrupa varios eventos relacionados del mismo actor o subsistema	VentasController maneja registrar, cancelar y consultar pedidos

La primera variante suele dar controllers más pequeños y con responsabilidad más clara; la segunda reduce la cantidad de clases pero, como se verá más adelante en la sesión, puede crecer demasiado si se le agregan casos de uso sin control.

CONCEPTO

Un Controller bien delimitado

```
class PedidoController:
    def __init__(self, repositorio_pedidos, servicio_inventario):
        self.repositorio = repositorio_pedidos
        self.inventario = servicio_inventario

    def registrar_pedido(self, datos_evento):
        pedido = Pedido.crear(datos_evento)          # delega en el experto (Creator)
        self.inventario.reservar(pedido.items)       # delega en otro experto
        self.repositorio.guardar(pedido)             # delega la persistencia
        return pedido.id
```

`PedidoController` recibe el evento de sistema ("registrar pedido") y **orquesta** la operación, pero no contiene lógica de interfaz (no sabe qué botón lo llamó) ni lógica de persistencia (no sabe cómo se guarda un pedido en la base de datos) — solo coordina, delegando cada tarea a la clase experta correspondiente.

CONTRAEJEMPLO

El Controller gordo (God Controller)

```
class VentasController:
    def registrar_pedido(self, datos_evento):
        # valida campos del formulario HTML directamente
        if not datos_evento.get("cliente_id"): raise ...
        # arma el SQL a mano
        conexion.execute("INSERT INTO pedidos VALUES (...)")
        # calcula impuestos con reglas de negocio embebidas
        total = sum(i.precio for i in datos_evento["items"]) * 1.19
        # arma el HTML de la respuesta
        return f"<div>Pedido creado, total: {total}</div>"
```

Este `VentasController` acumuló validación de formulario, acceso directo a base de datos, reglas de negocio (el cálculo de impuestos) y hasta generación de HTML — todo en un solo método. Es un **Controller gordo** (o *God Controller*): un antipatrón donde el controller deja de *coordinar* y pasa a *hacerlo todo* él mismo. Es la misma idea del **God Object** visto en la semana 7, aplicada específicamente a la capa de control.

CONCEPTO

Cómo evitarlo: el Controller delega, no ejecuta

El Controller gordo aparece cuando se le asignan responsabilidades que **ya tienen dueño** según otros patrones GRASP: la validación y el cálculo de impuestos son trabajo del **Information Expert** (la clase `Pedido`, que tiene los datos para calcularlo), la creación del objeto es trabajo de **Creator**, y el acceso a datos es trabajo de un repositorio dedicado.

- **Señal de alerta:** un controller con métodos de más de 15-20 líneas, o que importa librerías de base de datos, de HTML o de validación de formularios directamente.
- **Corrección:** extraer esa lógica a la clase que le corresponde por Information Expert o Creator, y dejar en el controller solo la secuencia de llamadas — como en el ejemplo de `PedidoController` de la diapositiva anterior.

TRANSICIÓN

De quién maneja el evento, a cómo se conectan las clases entre sí

Controller responde "¿quién recibe la orden?". Los dos patrones que siguen responden una pregunta distinta: una vez que las responsabilidades están repartidas entre varias clases (como en `PedidoController` , `Pedido` y el repositorio), **¿qué tan bien está hecho ese reparto?** Eso es lo que miden Low Coupling y High Cohesion.

CONCEPTO · RETOMANDO LA SEMANA 7

Low Coupling como patrón GRASP

En la semana 7 (Cohesión y Acoplamiento) vimos el **acoplamiento** como la medida de cuánto depende una clase de otras. Larman lo formaliza como patrón: **Low Coupling** asigna responsabilidades de forma que el acoplamiento entre clases se mantenga bajo, para que un cambio en una clase no obligue a cambiar muchas otras.

Criterios concretos para evaluar el acoplamiento de una clase en el diseño:

- **Cuántas otras clases conoce** (por atributo, por parámetro, por tipo de retorno, por herencia).
- **Cuántos métodos distintos** llama de cada una de esas clases.
- **Qué tan estable** es lo que conoce: depender de una interfaz estable acopla menos que depender del detalle interno de otra clase concreta (esto es exactamente DIP, visto en la semana 9).

CONCEPTO

Reducir el acoplamiento: un refactor

```
## Antes: acoplamiento alto – conoce el detalle de MySQL
class ReporteVentas:
    def generar(self):
        cursor = ConexionMySQL().obtener_cursor()
        cursor.execute("SELECT * FROM pedidos")
        ...

## Después: acoplamiento bajo – conoce solo el contrato
class ReporteVentas:
    def __init__(self, repositorio: RepositorioPedidos):
        self.repositorio = repositorio

    def generar(self):
        pedidos = self.repositorio.obtener_todos()
        ...
```

En la versión "antes", `ReporteVentas` conoce el detalle de MySQL: si el equipo cambia de motor de base de datos, hay que editar esta clase. En la versión "después" solo conoce la interfaz `RepositorioPedidos` — el mismo mecanismo de inyección de dependencias que se vio con DIP en la semana 9, aplicado aquí para bajar el acoplamiento.

CONCEPTO · RETOMANDO LA SEMANA 7

High Cohesion como patrón GRASP

También en la semana 7 vimos la **cohesión** como qué tan relacionadas están las responsabilidades dentro de una misma clase. Como patrón GRASP, **High Cohesion** asigna responsabilidades de forma que las de una clase estén fuertemente relacionadas y enfocadas en un solo propósito.

Criterios concretos para evaluar la cohesión de una clase:

- **La prueba de la frase:** ¿se puede describir lo que hace la clase en una sola oración, sin usar la palabra "y"? Si hace falta "y", probablemente tiene más de una responsabilidad.
- **Relación entre sus métodos:** ¿todos los métodos usan la mayoría de los atributos de la clase, o hay grupos de métodos que solo usan un subconjunto y nunca se mezclan con el resto?
- **Nivel de abstracción:** mezclar operaciones de muy alto nivel (orquestrar un proceso) con operaciones de muy bajo nivel (formatear una cadena de texto) en la misma clase es señal de baja cohesión.

CONCEPTO

Cohesión baja vs. alta: el mismo caso

```
## Cohesión baja: reportes + envío de correo + cálculo de impuestos
class GestorDeVentas:
    def generar_reporte(self): ...
    def enviar_correo_a_cliente(self, msj): ...
    def calcular_impuestos(self, pedido): ...

## Cohesión alta: una responsabilidad por clase
class ReporteVentas:
    def generar(self): ...

class NotificadorCliente:
    def enviar(self, msj): ...

class CalculadorImpuestos:
    def calcular(self, pedido): ...
```

`GestorDeVentas` no pasa la prueba de la frase: hace reportes **y** envía correos **y** calcula impuestos. Al dividirlo en tres clases, cada una se describe sin "y" — y, como efecto colateral, cada clase también terminó con **menos** clases de las que depende: separar por cohesión casi siempre reduce el acoplamiento también.

CONCEPTO

Low Coupling y High Cohesion se refuerzan mutuamente

No es casualidad que el refactor de la diapositiva anterior haya mejorado ambos a la vez. En la mayoría de los diseños:

- Una clase con **responsabilidades mezcladas** (baja cohesión) suele necesitar conocer más clases externas para cumplir cada una de esas responsabilidades distintas (alto acoplamiento).
- Al dividirla en clases con **una sola responsabilidad** (alta cohesión), cada clase resultante necesita menos colaboradores para cumplir su propósito, más acotado (bajo acoplamiento).

Por eso Larman los presenta como pareja: **diseñar pensando en alta cohesión es, la mayoría de las veces, el camino más directo hacia el bajo acoplamiento.**

TENSIÓN CONCEPTUAL

El propio Controller puede volverse de baja cohesión

Los tres patrones de hoy pueden entrar en conflicto entre sí. Un **Controller de fachada** (diapositiva 5) agrupa a propósito varios eventos relacionados en una sola clase para reducir la cantidad de controllers — pero si se le siguen agregando casos de uso sin relación real entre sí, esa misma clase deja de pasar la **prueba de la frase** de High Cohesion.

No hay una regla absoluta: un Controller de fachada bien delimitado (eventos genuinamente relacionados, como todo lo de "gestión de un pedido") mantiene alta cohesión; uno que agrupa eventos de módulos distintos "para no crear más clases" la pierde. La decisión se evalúa caso por caso, con los criterios ya vistos.



INTERACCIÓN

Evalúen un diseño con los tres patrones de hoy

En el chat, para el siguiente diseño respondan: ¿quién debería ser el Controller?, ¿dónde ven acoplamiento alto?, ¿qué clase no pasa la prueba de la frase?

1. Un botón "Confirmar pago" en la interfaz web ejecuta directamente la validación de la tarjeta, calcula el total con impuestos, y guarda el registro en la base de datos.
2. Una clase `Utilidades` del sistema tiene métodos para formatear fechas, enviar correos, validar contraseñas y generar reportes en PDF.

5 minutos · respondan en el chat

SÍNTESIS

Los tres patrones de hoy, en una frase

1. **Controller:** el evento de sistema lo maneja una clase de caso de uso, de sesión o del sistema — nunca la interfaz de usuario.
2. **Controller gordo:** antipatrón cuando el controller ejecuta la lógica en vez de delegarla al Information Expert o Creator correspondiente.
3. **Low Coupling:** minimizar cuántas clases conoce cada clase y qué tan atada está a sus detalles internos.
4. **High Cohesion:** cada clase debe describirse en una frase sin "y" — una responsabilidad, bien enfocada.
5. Los tres pueden entrar en tensión entre sí: un buen diseño los balancea caso por caso, no aplica ninguno "a ciegas".



CIERRE

Para el viernes: los últimos cuatro patrones GRASP

Con Information Expert y Creator (martes) y Controller, Low Coupling y High Cohesion (hoy), quedan cinco de los nueve patrones GRASP. **El viernes cerramos la serie completa** con los cuatro restantes: **Polymorphism**, **Pure Fabrication**, **Indirection** y **Protected Variations** — los patrones más orientados a manejar variación y cambio en el diseño.

Antes de esa sesión, revisen el diagrama de clases de su propio proyecto: identifiquen quién actúa hoy como Controller y busquen una clase que no pase la prueba de la frase de High Cohesion.

